Research Article

LG-CARL: Line-Graph Congestion-Aware Reinforcement Learning for Adaptive Routing

Xiangyi Liu[a,1]

[a]*Course Project in Computer Networks and Machine Learning*

Compiled on May 15, 2026

Abstract

This report presents LG-CARL, a congestion-aware adaptive routing system that combines directed line-graph representation learning with deep reinforcement learning. The project models each directed network link as a node in a line graph, applies a GraphSAGE-style encoder to link-level state features, and uses a DQN agent to select among candidate end-to-end paths for each traffic demand. The implementation includes a lightweight network simulator, synthetic and real-topology experiments, baseline policies, training and evaluation scripts, and route replay visualizations. Experiments show that LG-CARL learns stable routing behavior and performs close to shortest-path and dynamic-shortest-path routing under regular traffic, while the heavy-congestion scenario remains challenging and exposes directions for future reward and training improvements.

Keywords: *Adaptive routing, line graph, graph neural network, DQN, congestion-aware routing, network simulation*

■ 中文版本

■ 摘要

本文介绍LG-CARL，一个结合directed line graph、GraphSAGE和DQN 的拥塞感知自适应路由实验系统。系统将每条有向链路建模为line graph 中的节点，使用GNN 编码链路级状态，并通过DQN 在候选端到端路径中选择路由。实验表明，在常规流量场景下，LG-CARL 能够学习到接近shortest path 和dynamic shortest path 的路由行为；在强拥塞场景下，模型虽然比随机路由更好，并表现出更强的负载均衡倾向，但在时延、丢包和吞吐方面仍落后于dynamic shortest path。该结果说明项目实现完整且具有解释价值，同时也暴露出reward 设计和拥塞训练设置仍需改进。

关键词： 自适应路由；链路图；图神经网络；DQN；拥塞感知；网络仿真。

1. 引言

LG-CARL 是一个面向计算机网络动态路由问题的实验系统。传统最短路算法主要依据静态链路代价进行选路，在网络负载变化、队列积压和链路拥塞出现时，单纯依赖静态距离并不一定能得到低时延或低丢包的路径。本项目将路由问题建模为强化学习任务：环境不断产生流量请求，智能体根据当前链路状态在若干候选路径中选择一条路由，并通过时延、丢包、拥塞和路径切换等反馈学习策略。

本项目的核心思想是将原始网络拓扑转换为directed line graph。原图中的一条有向链路在line graph 中成为一个节点；如果两条有向链路可以连续转发，则在line graph 中连接它们。这样，GNN 的消息传递发生在链路状态之间，而不是只发生在路由器节点之间。由于拥塞本质上更接近链路级现象，例如链路利用率、队列长度、丢包率和链路延迟，因此line graph 表示更适合表达路由决策所需的动态状态。

项目目标不是证明模型在所有场景下都优于传统启发式方法，而是实现一个完整、可运行、可解释的GNN + RL 路由实验框架。该框架包括网络仿真器、候选路径生成、Line-Graph GraphSAGE 编码器、path-level DQN、baseline 对比、实验结果表格和可视化GIF。

贡献

本文主要贡献如下：

- 提出并实现一个以链路为中心的自适应路由实验系统，将directed line graph 表示、GraphSAGE 编码器和DQN 路径选择结合起来。
- 构建轻量网络仿真器，能够模拟链路负载、队列、利用率、时延、丢包和服务衰减过程。
- 提供Random-K、Shortest Path 和Dynamic Shortest Path baseline，用于定量比较学习式路由策略。
- 在NSFNet-like 拓扑和SNDlib Polska 真实拓扑上进行常规流量和拥塞流量实验，并给出可视化replay 分析。

2. 相关工作

2.1. 传统路由与流量工程

早期互联网路由协议主要依赖分布式控制和静态或半静态链路代价。RIP [1] 和OSPF [6] 代表了经典的距离向量和链路状态路由协议，ECMP [8] 进一步通过等价路径分摊流量。MPLS [10] 和RSVP-TE [9] 则为显式路径控制和流量工程提供了协议基础。这些机制稳定、可解释且工程成熟，但通常需要人工配置权重或策略，对快速变化的负载和拥塞状态适应能力有限。

为了提升网络资源利用率，流量工程研究开始从单纯最短路转向全局优化。Fortz 和Thorup 研究了通过优化OSPF 权重实现Internet traffic engineering [7, 11]；Applegate 等研究了在流量矩阵变化和不确定性下的鲁棒intra-domain routing [13]；Roughan 等对骨干网流量变化进行了测量建模[12]；Kandula 等进一步讨论了响应性和稳定性之间的权衡[14]。这些工作说明路由优化不仅要考虑最短路径，还要考虑负载均衡、鲁棒性和稳定性。

SDN 进一步将路由控制从分布式协议推向集中式、可编程控制。OpenFlow [15]、DevoFlow [18]、Hedera [17]、Google B4 [20] 和SWAN [19] 展示了集中式控制在数据中心和广域网中的潜力。Kreutz 等[22] 和Akyildiz 等[21] 对SDN 架构和流量工程进行了系统综述。与这些优化和控制平面工作相比，LG-CARL 关注的是在一个可控仿真环境中学习候选路径选择策略，并通过reward 将拥塞、丢包和稳定性纳入统一目标。

2.2. 强化学习路由与学习式网络控制

强化学习为动态路由提供了一种无需显式解析模型的自适应优化思路。Q-learning [2] 和现代强化学习框架[34] 为此类方法奠定了理论基础。Boyan 和Littman 的Q-routing [3] 是较早将强化学习用于动态网络路由的代表性工作；Ant-based routing [4, 5] 则从群体智能角度探索了自适应路径选择。深度学习兴起

Contact: lnubruni@gmail.com.

This report documents a runnable implementation of LG-CARL, including the simulator, GNN-DQN routing agent, baseline policies, quantitative evaluation, and route replay visualizations. The report is organized with the Chinese version first and the English version second.

后，DQN [23]、Double DQN [24]、Prioritized Experience Replay [26] 和Dueling Network [27] 使基于值函数的控制能够处理更复杂的状态空间。

学习式网络控制也被用于资源管理、拥塞控制、视频传输和流量工程。DeepRM [25] 通过深度强化学习学习资源调度；Pensieve [31] 将RL 用于自适应码率控制；Aurora [38] 和Classic Meets Modern [40] 探索了学习式拥塞控制；Valadarsky 等[32] 和Xu 等[36] 则讨论了深度强化学习在路由和网络控制中的应用。与这些工作相比，本项目使用强化学习学习path-level 选择，但强调line graph 表示，使链路级拥塞状态成为策略输入的核心。

2.3. 图神经网络与网络建模

图神经网络为处理拓扑结构数据提供了通用工具。Scarselli 等[16] 提出了早期GNN 框架，随后GCN [30]、GraphSAGE [29]、GAT [35]、MPNN [28] 和GIN [39] 推动了图表示学习的发展。Battaglia 等[33] 从relational inductive bias 角度总结了graph networks 的表达能力。这些工作共同说明，当问题天然具有图结构时，显式利用拓扑关系通常比普通MLP 更有归纳偏置。

通信网络是GNN 的自然应用场景。RouteNet [41] 证明GNN 可以建模拓扑、路由和流量之间的关系，并预测delay、jitter 和loss 等网络KPI。RouteNet-Fermi [43] 进一步提升了网络性能建模能力。Suarez-Varela 等[44] 系统总结了GNN 在通信网络中的应用场景和机会；IGNNITION [42] 则试图降低网络系统中构建GNN 原型的门槛。Almasan 等[37] 将GNN 与DRL agent 结合，用于提升路由优化在不同拓扑上的泛化能力；Abrol 等[45] 使用DGCNN 和DQN 进行自适应流量路由。这些研究与LG-CARL 最为接近，但本项目采用directed line graph，使GNN 的节点直接对应链路而非路由器，从而更强调链路拥塞状态的传播建模。

2.4. 与本工作的关系

综上，传统流量工程提供了强baseline 和可解释优化目标，强化学习提供了动态决策框架，GNN 提供了拓扑泛化和结构表示能力。LG-CARL 将这三条线索结合：用line graph 表示链路级状态，用GraphSAGE 学习链路嵌入，用DQN 在候选路径集合上进行决策。与完全端到端的黑盒路由不同，本项目保留了k-shortest candidate paths，使动作空间可控且便于可视化解释；与传统shortest path 相比，它能够将拥塞、丢包和切换稳定性纳入学习目标。

3. 方法

3.1. 网络状态建模

原始网络被表示为有向图$G = (V, E)$，其中V 是路由器集合，E 是有向链路集合。每条链路$e = (u, v)$ 具有容量、基础延迟、当前负载、队列长度、利用率、丢包率和故障状态等属性。

为了更直接地建模链路之间的拥塞传播关系，项目构建directed line graph $G_L = (V_L, E_L)$。其中V_L 中的每个节点对应原图中的一条有向链路。如果$e_1 = (u, v)$ 与$e_2 = (v, w)$ 可以连续转发，则在line graph 中加入边$e_1 \to e_2$。该设计使GNN 能够在相邻链路之间聚合信息，从而学习局部拥塞模式。

3.2. 候选路径动作空间

每一步环境产生一个流量请求：

$$f_t = (s_t, d_t, b_t), \tag{1}$$

其中s_t 是源节点，d_t 是目的节点，b_t 是需求大小。系统首先通过k-shortest paths 生成K 条候选路径：

$$\mathcal{P}_{s,d} = \{p_1, p_2, \dots, p_K\}. \tag{2}$$

智能体并不是在所有可能路径中任意搜索，而是在有限候选路径中评分和选择。这种设计降低了动作空间复杂度，同时保证候选路径本身具有基本可行性。

3.3. Path-Level DQN

模型先使用GraphSAGE 风格的line-graph 编码器得到每条链路的嵌入表示。对某条候选路径p_k，模型聚合路径上所有链路嵌入，并拼接路径特征、源节点嵌入、目的节点嵌入和需求大小，最后通过MLP 输出该路径的Q 值：

$$Q(s_t, p_k) = \mathrm{MLP}\left(h(p_k), x(p_k), e_s, e_d, b_t\right). \tag{3}$$

评估阶段选择Q 值最高的有效候选路径：

$$p^* = \arg\max_{p_k \in \mathcal{P}_{s,d}} Q(s_t, p_k). \tag{4}$$

3.4. 奖励函数

奖励函数采用负代价形式，主要惩罚以下因素：路径时延、丢包、最大链路利用率、路径切换代价、关键链路风险和非法动作。概念上可以写为：

$$r_t = -\left(\alpha D_t + \beta L_t + \gamma U_t + \lambda S_t + \eta R_t\right), \tag{5}$$

其中D_t 表示归一化时延，L_t 表示丢包，U_t 表示最大链路利用率，S_t 表示路径切换代价，R_t 表示关键链路风险。由于reward 是负代价，所以数值越接近0 表示越好。

4. 实现

项目使用Python、PyTorch 和NetworkX 实现。主要模块包括：

- **Routing environment**: 负责生成流量请求、维护网络状态、执行路由动作并返回reward。
- **Network simulator**: 维护链路load、queue、utilization、delay 和loss。
- **Line graph utilities**: 将原始拓扑转换为directed line graph，并生成链路特征。
- **DQN agent**: 使用replay buffer、target network 和epsilon-greedy 策略进行训练。
- **Baselines**: 包括Random-K、Shortest Path 和Dynamic Shortest Path。
- **Visualization**: 输出训练曲线、评估柱状图、拓扑图和route replay GIF。

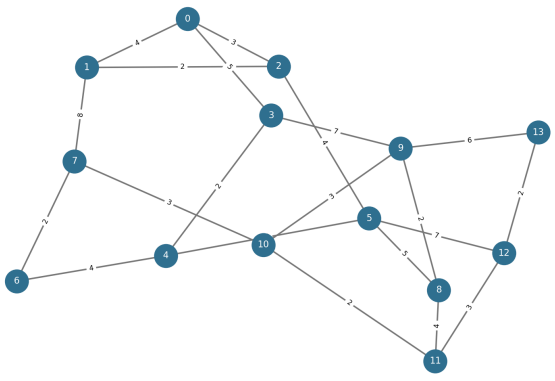

Figure 1. NSFNet-like topology used by the default experiment.

5. 实验设置

实验覆盖三类主要配置。第一类是默认NSFNet-like 拓扑，使用合成uniform traffic；第二类是SNDlib Polska 真实拓扑和归一化demand matrix；第三类是更高需求、更慢服务率的SNDlib Polska congested 配置，用作拥塞压力测试。

评价指标包括：

Table 1. Selected evaluation results. Lower delay/loss/utilization is better; higher throughput is better.

Scenario	Method	Avg Delay	Loss	Throughput	Max Util.	Balance Std
NSFNet uniform	Shortest Path	9.778	0.000	5.467	0.217	0.0196
NSFNet uniform	Dynamic Shortest	9.777	0.000	5.467	0.217	0.0196
NSFNet uniform	LG-CARL	10.880	0.000	5.467	0.193	0.0205
SNDlib Polska regular	Shortest Path	4.512	0.000	5.709	0.167	0.0154
SNDlib Polska regular	Dynamic Shortest	4.512	0.000	5.709	0.167	0.0154
SNDlib Polska regular	LG-CARL	4.517	0.000	5.709	0.167	0.0153
SNDlib Polska congested	Shortest Path	464.048	0.561	25.737	1.500	0.6123
SNDlib Polska congested	Dynamic Shortest	439.005	0.617	22.454	1.500	0.4835
SNDlib Polska congested	LG-CARL	699.294	0.726	16.060	1.500	0.2976

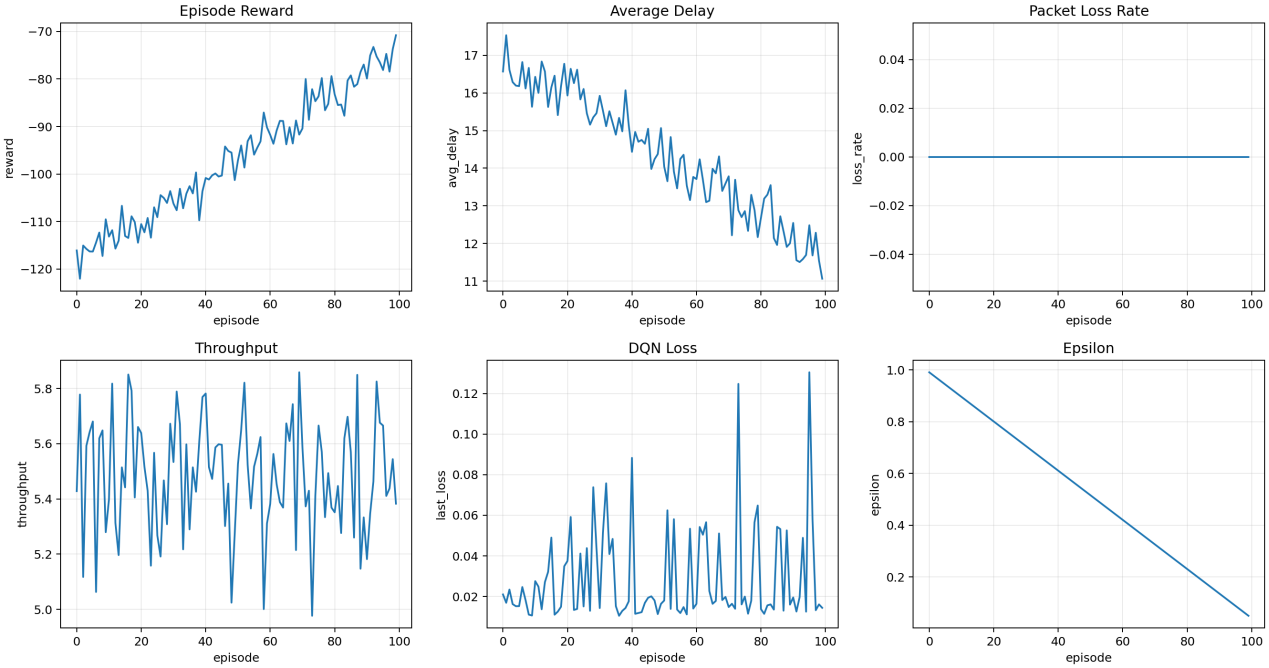

Figure 2. Training curves for reward, average delay, packet loss rate, throughput, DQN loss, and epsilon.

- **Average delay**: 每一步路由后的平均路径时延，越低越好。
- **Loss rate**: 丢弃流量占总需求的比例，越低越好。
- **Throughput**: 每一步平均成功交付的流量，越高越好。
- **Max utilization**: episode 中最大链路利用率，越低通常表示瓶颈压力越小。
- **Load balance std**: 链路利用率标准差，越低表示负载分布越均衡。
- **Switch rate**: 路径切换比例，用于衡量路由稳定性。

6. 实验结果

从Table 1 可以看到，在默认NSFNet uniform 场景中，LG-CARL 的平均时延为10.880，高于dynamic shortest path 的9.777，但最大链路利用率从0.217 降至0.193，说明模型倾向于一定程度上分散链路负载。在SNDlib Polska regular 场景中，LG-CARL 的平均时延4.517 与shortest path 的4.512 几乎相同，loss rate 也保持为0，说明常规负载下模型可以学习到接近强baseline 的行为。

在congested 场景中，LG-CARL 相比random routing 有明显改善，但仍落后于dynamic shortest path。其平均时延为699.294，高于dynamic shortest path 的439.005；loss rate 为0.726，也高于shortest path 的0.561。与此同时，LG-CARL 的load balance std 为0.2976，低于shortest path 和dynamic shortest path，说明模型确实更均匀地分散了负载，但这种分散没有转化为更低的丢包和更高的吞吐。

7. 可视化分析

为了让模型行为更容易解释，项目实现了route replay 可视化。图中节点表示路由器，边表示链路；边颜色越接近红色表示链路利用率越高；蓝白色高亮线表示LG-CARL 当前选择的路径。右侧面板显示当前flow、demand、delay、loss、delivered traffic 和max utilization。

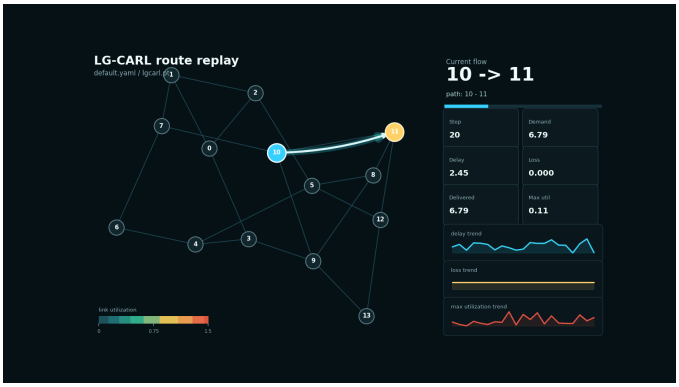

Figure 4. Route replay frame under regular traffic. The selected path is highlighted in cyan.

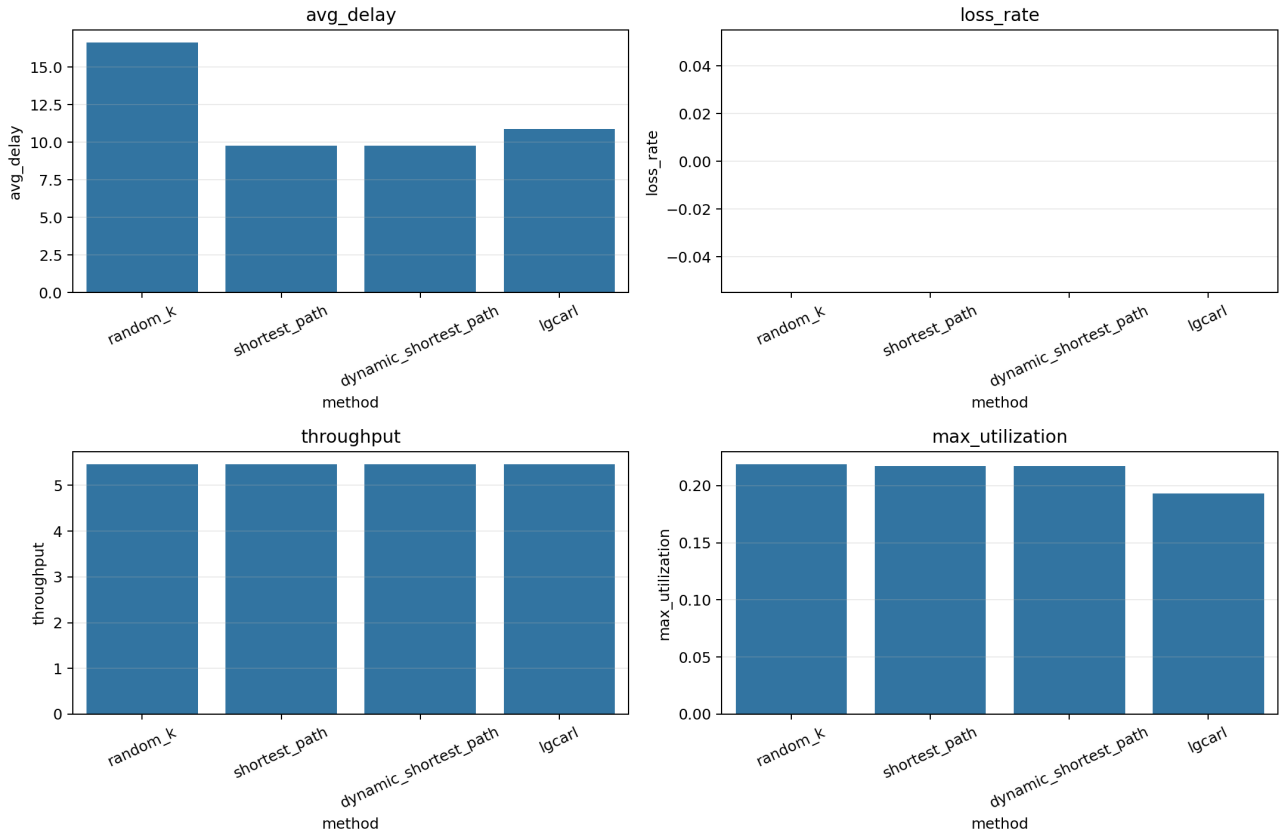

Figure 3. Evaluation bar charts comparing LG-CARL with routing baselines.

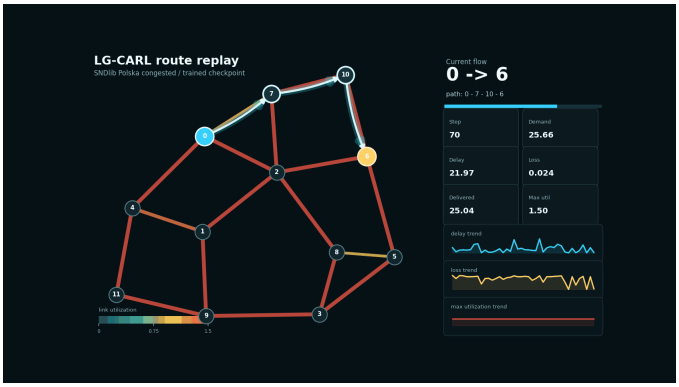

Figure 5. Route replay frame under heavy congestion. Many links become orange or red, and packet loss appears.

常规场景中，大部分链路保持冷色，loss 基本为0。拥塞场景中，多条链路达到高利用率，max utilization 经常顶到1.5，并且loss trend 出现波动。这说明GIF 更适合用于解释模型行为和网络状态变化，而不是单独证明模型优于baseline。最终性能仍应以Table 1 中的定量指标为准。

8. 结论

本项目完成了一个可运行的GNN + RL 自适应路由实验系统。LG-CARL 能够将链路级状态编码到directed line graph 中，并通过DQN 在候选路径中选择路由。在常规流量下，模型能达到接近shortest path 和dynamic shortest path 的性能；在部分指标上，例如最大链路利用率或负载均衡，它表现出分散负载的倾向。

然而，在强拥塞场景下，当前策略尚未超过dynamic shortest path。实验结果表明，模型虽然学习到了负载均衡行为，但该行为没有充分转化为更低的丢包率和更高的吞吐量。这说明reward 权重、训练稳定性、拥塞数据分布和强化学习算法仍有进一步优化空间。

总体而言，LG-CARL 作为课程项目是完整的：它包含明确的问题建模、可运行代码、多baseline 对比、真实拓扑支持、定量分析和可视化解释。未来可以继续加入reward ablation、double DQN、多随机种子置信区间和未见拓扑泛化实验。

English Version

Abstract

This report presents LG-CARL, a congestion-aware adaptive routing system that combines directed line graphs, GraphSAGE, and DQN. The system models each directed link as a node in a line graph, uses a GNN to encode link-level states, and applies a DQN policy to select among candidate end-to-end paths. Experiments show that LG-CARL learns behavior close to shortest-path and dynamic-shortest-path routing under regular traffic. Under heavy congestion, it improves over random routing and shows stronger load-balancing behavior, but it still underperforms dynamic shortest path in delay, loss, and throughput. The result demonstrates a complete and interpretable implementation while exposing reward design and congestion training as important future directions.

Keywords: adaptive routing; line graph; graph neural network; DQN; congestion-aware routing; network simulation.

1. Introduction

LG-CARL is an experimental system for adaptive routing in computer networks. Classical shortest-path routing mainly relies on static link costs. When network load changes dynamically, queues grow, and bottleneck links become congested, the shortest static path may no longer provide low delay or low packet loss. This project formulates routing as a reinforcement learning problem: the environment generates traffic demands, the agent selects one route from several candidate paths, and the selected action is evaluated through delay, loss, congestion, and switching-related feedback.

The key idea of this project is to represent the network as a directed line graph. In the original graph, routers are nodes and directed links are edges. In the line graph, each directed link becomes a node. If two directed links can be used consecutively for forwarding, an edge is created between their corresponding line-graph nodes. This makes message passing occur between link states rather than only between router nodes. Since congestion is primarily a link-level phenomenon, this representation is suitable for modeling utilization, queue length, loss rate, and link delay.

The goal of the project is not to claim that the learned policy dominates all routing heuristics in every setting. Instead, it aims to provide a complete, runnable, and interpretable GNN + RL routing framework. The framework includes a network simulator, candidate path generation, a Line-Graph GraphSAGE encoder, a path-level DQN agent, baseline comparisons, quantitative tables, and route replay visualizations.

Contributions

The main contributions are:

- We implement a link-centric adaptive routing system that combines directed line-graph representation, a GraphSAGE encoder, and DQN-based path selection.
- We build a lightweight simulator that updates link load, queue, utilization, delay, loss, and service decay.
- We include Random-K, Shortest Path, and Dynamic Shortest Path baselines for quantitative comparison.
- We evaluate the system on an NSFNet-like topology and the SNDlib Polska topology under both regular and congested traffic, with route replay visualizations for interpretation.

2. Related Work

2.1. Classical Routing and Traffic Engineering

Early Internet routing protocols rely on distributed control and static or semi-static link costs. RIP [1] and OSPF [6] are representative distance-vector and link-state protocols, while ECMP [8] distributes traffic over equal-cost paths. MPLS [10] and RSVP-TE [9] provide protocol-level support for explicit path control and traffic engineering. These mechanisms are stable, interpretable, and mature, but they often require manually configured weights or policies and may adapt slowly to rapidly changing load.

Traffic engineering research moved beyond pure shortest-path routing toward global optimization. Fortz and Thorup studied OSPF weight optimization for Internet traffic engineering [7, 11]. Applegate et al. investigated robust intra-domain routing under changing and uncertain traffic matrices [13]. Roughan et al. measured and modeled backbone traffic variability [12], and Kandula et al. studied the trade-off between responsiveness and stability [14]. These works show that routing optimization must account for load balancing, robustness, and stability, not merely path length.

Software-defined networking further pushed routing control toward centralized and programmable systems. OpenFlow [15], DevoFlow [18], Hedera [17], Google B4 [20], and SWAN [19] demonstrate the potential of centralized control in data-center and wide-area networks. Kreutz et al. [22] and Akyildiz et al. [21] provide broader surveys of SDN architectures and traffic engineering. Compared with these optimization and control-plane systems, LG-CARL focuses on learning a candidate-path selection policy in a controllable simulator, using reward terms to combine congestion, loss, and routing stability.

2.2. Reinforcement Learning for Routing and Network Control

Reinforcement learning provides an adaptive optimization framework that does not require a fully specified analytic model of the network. Q-learning [2] and the general RL framework [34] form the foundation for such methods. Q-routing by Boyan and Littman [3] is one of the earliest examples of reinforcement learning for dynamic packet routing. Ant-based routing [4, 5] explored adaptive path selection from a swarm-intelligence perspective. With deep learning, DQN [23], Double DQN [24], Prioritized Experience Replay [26], and Dueling Networks [27] made value-based control more suitable for high-dimensional states.

Learning-based network control has also been applied to resource management, congestion control, adaptive video streaming, and traffic engineering. DeepRM [25] uses deep reinforcement learning for resource scheduling. Pensieve [31] applies RL to adaptive bitrate streaming. Aurora [38] and Classic Meets Modern [40] explore learning-based congestion control. Valadarsky et al. [32] and Xu et al. [36] discuss deep reinforcement learning for routing and network control. LG-CARL follows this learning-based direction but emphasizes line-graph state representation, making link-level congestion the central policy input.

2.3. Graph Neural Networks and Network Modeling

Graph neural networks provide a general framework for learning from topological data. Scarselli et al. [16] introduced an early GNN formulation. Later models such as GCN [30], GraphSAGE [29], GAT [35], MPNN [28], and GIN [39] advanced graph representation learning. Battaglia et al. [33] summarized graph networks from the perspective of relational inductive bias. These works show that when a problem is naturally graph-structured, explicitly exploiting topology can provide a useful inductive bias compared with plain MLPs.

Communication networks are a natural application domain for GNNs. RouteNet [41] showed that GNNs can model the relationship among topology, routing, and traffic and predict network KPIs such as delay, jitter, and loss. RouteNet-Fermi [43] further improved GNN-based network performance modeling. Suarez-Varela et al. [44] surveyed opportunities for GNNs in communication networks, and IGNNITION [42] lowered the barrier to prototyping GNNs for networking systems. Almasan et al. [37] combined GNNs with a DRL agent to improve routing optimization across different topologies, while Abrol et al. [45] used DGCNN and DQN for adaptive traffic

routing. LG-CARL is closest to this line of work, but it uses a directed line graph so that GNN nodes directly correspond to links rather than routers, emphasizing link-level congestion propagation.

2.4. Position of This Work

Overall, classical traffic engineering provides strong baselines and interpretable objectives, reinforcement learning provides a dynamic decision-making framework, and GNNs provide topology-aware representation learning. LG-CARL combines these directions: it represents link-level state with a line graph, learns link embeddings with GraphSAGE, and uses DQN to select among candidate paths. Unlike fully black-box routing, it preserves k-shortest candidate paths, keeping the action space controlled and visualizable. Unlike pure shortest-path routing, it can include congestion, loss, and switching stability in the learning objective.

3. Method

3.1. Network State Modeling

The physical network is represented as a directed graph $G = (V, E)$, where V is the set of routers and E is the set of directed links. Each link $e = (u, v)$ has capacity, base delay, current load, queue length, utilization, loss rate, and failure state.

To model congestion propagation between adjacent links, the project constructs a directed line graph $G_L = (V_L, E_L)$. Each node in V_L corresponds to one directed link in the original graph. If $e_1 = (u, v)$ and $e_2 = (v, w)$ can be used consecutively in a forwarding path, the line graph contains the edge $e_1 \rightarrow e_2$. This allows the GNN to aggregate information across adjacent links and learn local congestion patterns.

3.2. Candidate Path Action Space

At each step, the environment generates a traffic demand:

$$f_t = (s_t, d_t, b_t), \tag{6}$$

where s_t is the source node, d_t is the destination node, and b_t is the demand size. The system first generates K candidate paths using k-shortest paths:

$$\mathcal{P}_{s,d} = \{p_1, p_2, \dots, p_K\}. \tag{7}$$

The agent does not search over all possible paths. It scores and selects from a bounded set of feasible candidate paths. This design reduces the action-space complexity while keeping the selected action interpretable as an end-to-end route.

3.3. Path-Level DQN

The model first applies a GraphSAGE-style encoder on the directed line graph to produce link embeddings. For each candidate path p_k, the model aggregates the embeddings of the links on that path and concatenates path-level features, source and destination embeddings, and the demand size. A multilayer perceptron then outputs a Q-value for the candidate path:

$$Q(s_t, p_k) = \mathrm{MLP}\left(h(p_k), x(p_k), e_s, e_d, b_t\right). \tag{8}$$

During evaluation, the policy selects the valid candidate path with the largest Q-value:

$$p^* = \arg\max_{p_k \in \mathcal{P}_{s,d}} Q(s_t, p_k). \tag{9}$$

3.4. Reward Function

The reward is implemented as a negative cost. The agent is penalized for high path delay, packet loss, high maximum link utilization, frequent route switching, critical-link risk, and invalid actions. Conceptually:

$$r_t = -\left(\alpha D_t + \beta L_t + \gamma U_t + \lambda S_t + \eta R_t\right), \tag{10}$$

where D_t is normalized delay, L_t is packet loss, U_t is maximum link utilization, S_t is switching cost, and R_t is critical-link risk. Since reward is a negative cost, values closer to zero are better.

4. Implementation

The project is implemented in Python with PyTorch and NetworkX. The main modules are:

- **Routing environment**: generates traffic demands, maintains network state, executes routing actions, and returns rewards.
- **Network simulator**: updates link load, queue, utilization, delay, and loss.
- **Line graph utilities**: convert the original topology into a directed line graph and build link features.
- **DQN agent**: uses replay buffer, target network, and epsilon-greedy exploration.
- **Baselines**: include Random-K, Shortest Path, and Dynamic Shortest Path.
- **Visualization**: generates training curves, evaluation bars, topology plots, and route replay GIFs.

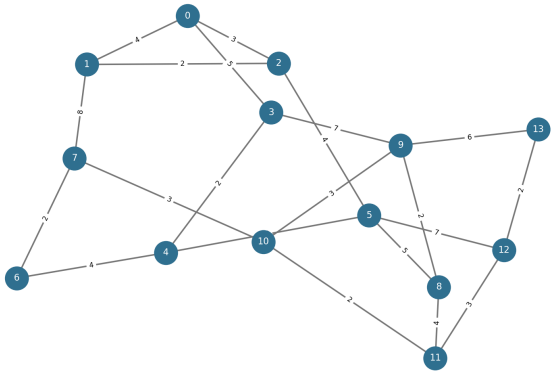

Figure 1. NSFNet-like topology used by the default experiment.

5. Experimental Setup

The experiments cover three major configurations. The first one uses an NSFNet-like topology with synthetic uniform traffic. The second one uses the SNDlib Polska topology with a normalized real demand matrix. The third one uses a congested SNDlib Polska configuration with larger demands and slower service, serving as a stress test.

The evaluation metrics are:

- **Average delay**: mean path delay after routing, lower is better.
- **Loss rate**: dropped traffic divided by total demand, lower is better.
- **Throughput**: average delivered traffic per step, higher is better.
- **Max utilization**: maximum link utilization within an episode, lower usually indicates less bottleneck pressure.
- **Load balance std**: standard deviation of link utilization, lower means more balanced traffic distribution.
- **Switch rate**: fraction of steps that change a route for the same source-destination pair.

Table 1. Selected evaluation results. Lower delay/loss/utilization is better; higher throughput is better.

Scenario	Method	Avg Delay	Loss	Throughput	Max Util.	Balance Std
NSFNet uniform	Shortest Path	9.778	0.000	5.467	0.217	0.0196
NSFNet uniform	Dynamic Shortest	9.777	0.000	5.467	0.217	0.0196
NSFNet uniform	LG-CARL	10.880	0.000	5.467	0.193	0.0205
SNDlib Polska regular	Shortest Path	4.512	0.000	5.709	0.167	0.0154
SNDlib Polska regular	Dynamic Shortest	4.512	0.000	5.709	0.167	0.0154
SNDlib Polska regular	LG-CARL	4.517	0.000	5.709	0.167	0.0153
SNDlib Polska congested	Shortest Path	464.048	0.561	25.737	1.500	0.6123
SNDlib Polska congested	Dynamic Shortest	439.005	0.617	22.454	1.500	0.4835
SNDlib Polska congested	LG-CARL	699.294	0.726	16.060	1.500	0.2976

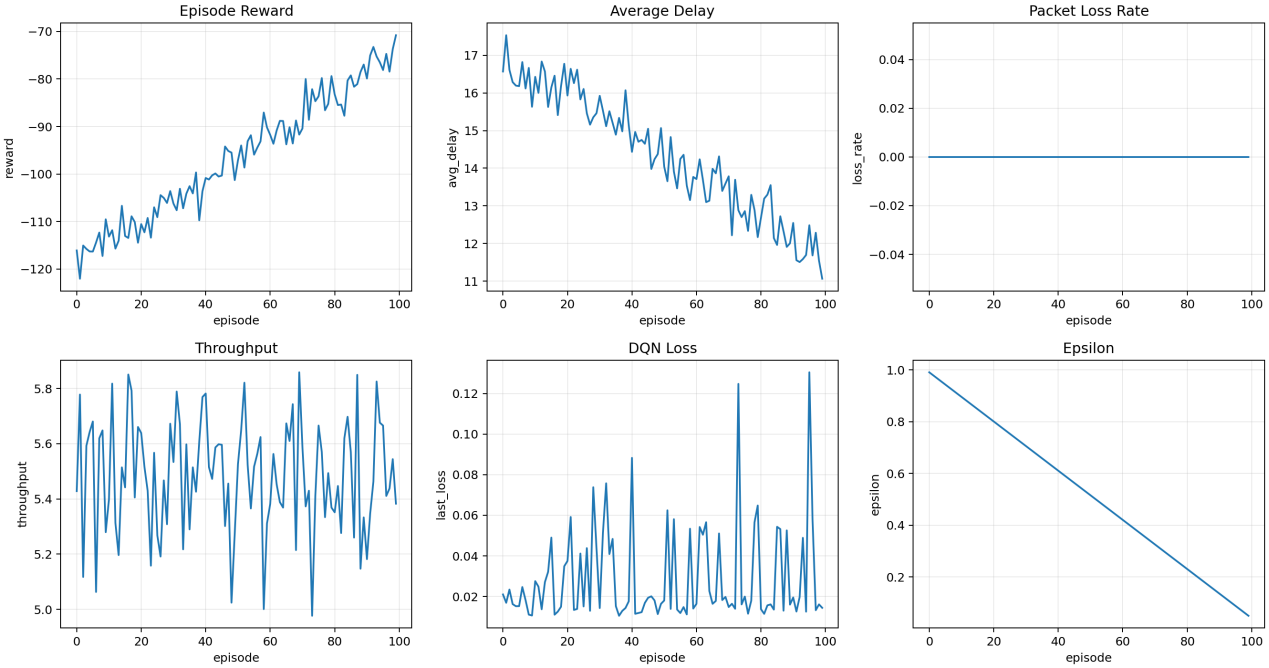

Figure 2. Training curves for reward, average delay, packet loss rate, throughput, DQN loss, and epsilon.

6. Results

As shown in Table 1, LG-CARL obtains an average delay of 10.880 in the NSFNet uniform setting, compared with 9.777 for dynamic shortest path. However, its maximum utilization is lower, decreasing from 0.217 to 0.193, which suggests that the learned policy tends to distribute traffic more broadly. In the SNDlib Polska regular setting, LG-CARL obtains an average delay of 4.517, nearly identical to the 4.512 achieved by shortest-path routing, while maintaining zero loss. This indicates that the model can learn behavior close to strong baselines under regular load.

In the congested setting, LG-CARL improves over random routing but still underperforms dynamic shortest path. Its average delay is 699.294, compared with 439.005 for dynamic shortest path, and its loss rate is 0.726, higher than the 0.561 of shortest path. At the same time, LG-CARL obtains the lowest load balance standard deviation, 0.2976. This means that the model learns to spread traffic more evenly, but this behavior does not yet translate into lower loss or higher throughput under heavy congestion.

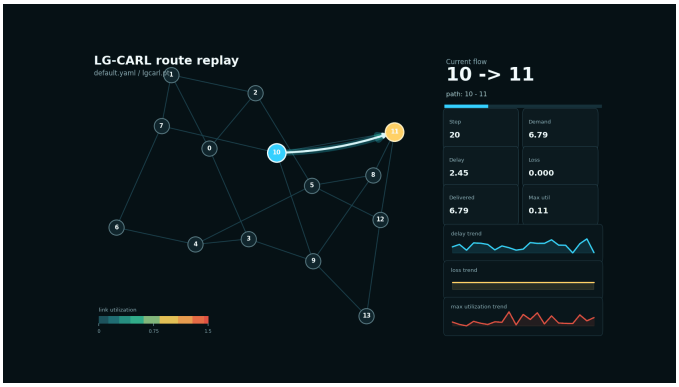

Figure 4. Route replay frame under regular traffic. The selected path is highlighted in cyan.

7. Visualization Analysis

To make the model behavior easier to interpret, the project implements route replay visualization. In the replay, nodes are routers and edges are network links. Warmer edges indicate higher link utilization, while the cyan highlighted route is the path selected by LG-CARL for the current flow. The right-side panel reports the current flow, demand, delay, loss, delivered traffic, and maximum utilization.

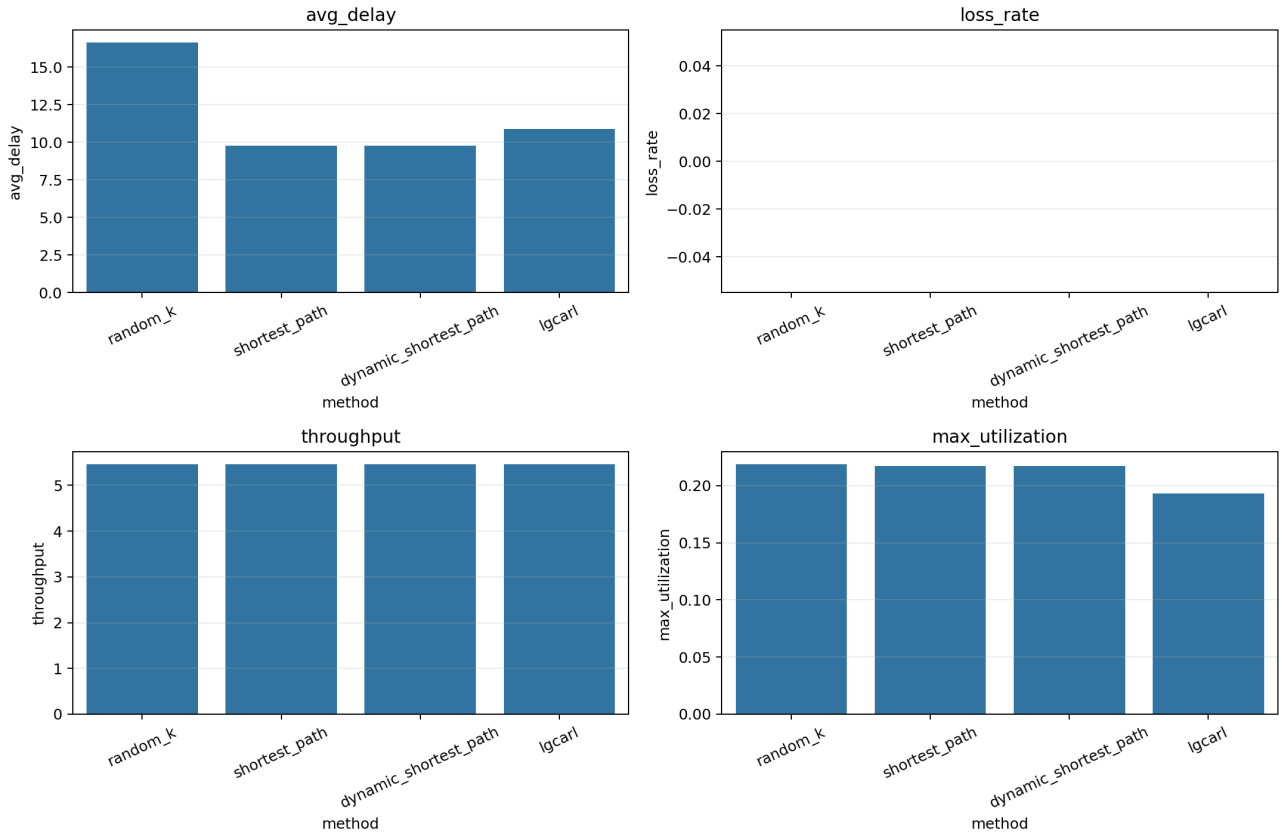

Figure 3. Evaluation bar charts comparing LG-CARL with routing baselines.

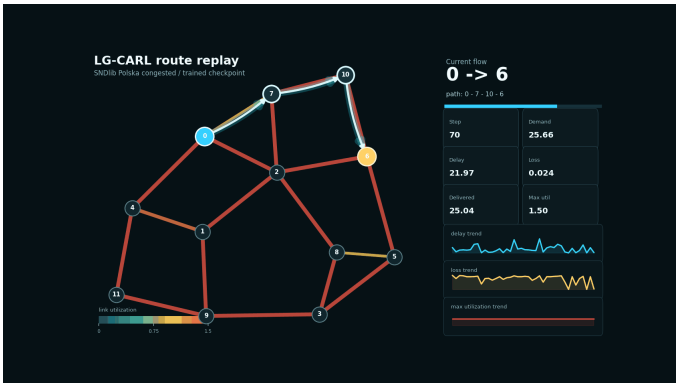

Figure 5. Route replay frame under heavy congestion. Many links become orange or red, and packet loss appears.

Under regular traffic, most links remain in cold colors and packet loss is nearly zero. Under congested traffic, multiple links reach high utilization, maximum utilization often saturates at 1.5, and the loss trend becomes visible. The GIFs are therefore useful for explaining policy behavior and network state changes. They should not be treated as standalone proof that the model outperforms a baseline; quantitative metrics such as those in Table 1 remain the primary evidence.

8. Conclusion

This project implements a runnable GNN + RL adaptive routing system. LG-CARL encodes link-level state through a directed line graph and uses DQN to choose among candidate end-to-end paths. Under regular traffic, the learned policy reaches performance close to shortest-path and dynamic-shortest-path baselines. In some metrics, such as maximum link utilization and load balance, it shows a tendency to spread traffic more evenly.

However, the learned policy does not yet outperform dynamic shortest path under heavy congestion. The results show that LG-CARL learns load-balancing behavior, but this behavior is not sufficient to reduce packet loss or increase throughput in the most difficult setting. This suggests that reward weights, training stability, traffic distributions, and the reinforcement learning algorithm itself remain important areas for future improvement.

Overall, LG-CARL is a complete course-scale project. It includes a clear problem formulation, runnable implementation, multiple baselines, real-topology support, quantitative analysis, and interpretable visualizations. Future extensions could include reward ablation, double DQN, confidence intervals over multiple seeds, and generalization experiments on unseen topologies.

Acknowledgment

The implementation and experiments were developed as a course project. The design is inspired by earlier work on Q-routing, graph neural network based network modeling, and deep reinforcement learning for control and routing [3, 23, 29, 37, 41].

References

[1] C. L. Hedrick, “Routing Information Protocol”, IETF, RFC 1058, 1988. [Online]. Available: https://www.rfc-editor.org/rfc/rfc1058.

[2] C. J. C. H. Watkins and P. Dayan, “Q-learning”, *Machine Learning*, vol. 8, no. 3, pp. 279–292, 1992. DOI: 10.1007/BF00992698.

[3] J. A. Boyan and M. L. Littman, “Packet routing in dynamically changing networks: A reinforcement learning approach”, in *Advances in Neural Information Processing Systems*, 1993. [Online]. Available: https://proceedings.neurips.cc/paper/1993/file/4ea06fbc83cdd0a06020c35d50e1e89a-Paper.pdf.

[4] R. Schoonderwoerd, O. Holland, J. Bruten, and L. Rothkrantz, "Ant-based load balancing in telecommunications networks", *Adaptive Behavior*, vol. 5, no. 2, pp. 169–207, 1996. DOI: 10.1177/105971239600500203.

[5] G. D. Caro and M. Dorigo, "AntNet: Distributed stigmergetic control for communications networks", *Journal of Artificial Intelligence Research*, vol. 9, pp. 317–365, 1998. DOI: 10.1613/jair.530.

[6] J. Moy, "OSPF Version 2", IETF, RFC 2328, 1998. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2328.

[7] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights", in *Proceedings IEEE INFOCOM*, 2000, pp. 519–528. DOI: 10.1109/INFCOM.2000.832225.

[8] C. E. Hopps, "Analysis of an Equal-Cost Multi-Path Algorithm", IETF, RFC 2992, 2000. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2992.

[9] D. O. Awduche, L. Berger, D.-H. Gan, T. Li, V. Srinivasan, and G. Swallow, "RSVP-TE: Extensions to RSVP for LSP Tunnels", IETF, RFC 3209, 2001. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3209.

[10] E. C. Rosen, A. Viswanathan, and R. Callon, "Multiprotocol Label Switching Architecture", IETF, RFC 3031, 2001. [Online]. Available: https://www.rfc-editor.org/rfc/rfc3031.

[11] B. Fortz, J. Rexford, and M. Thorup, "Traffic engineering with traditional IP routing protocols", *IEEE Communications Magazine*, vol. 40, no. 10, pp. 118–124, 2002. DOI: 10.1109/MCOM.2002.1039866.

[12] M. Roughan, A. Greenberg, C. Kalmanek, M. Rumsewicz, J. Yates, and Y. Zhang, "Experience in measuring backbone traffic variability: Models, metrics, measurements and meaning", in *Proceedings of the ACM SIGCOMM Internet Measurement Workshop*, 2002, pp. 91–92. DOI: 10.1145/637201.637216.

[13] D. Applegate and E. Cohen, "Making intra-domain routing robust to changing and uncertain traffic demands: Understanding fundamental tradeoffs", in *Proceedings of ACM SIGCOMM*, 2003, pp. 313–324. DOI: 10.1145/863955.863991.

[14] S. Kandula, D. Katabi, B. Davie, and A. Charny, "Walking the tightrope: Responsive yet stable traffic engineering", in *Proceedings of ACM SIGCOMM*, 2005, pp. 253–264. DOI: 10.1145/1080091.1080122.

[15] N. McKeown et al., "OpenFlow: Enabling innovation in campus networks", *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008. DOI: 10.1145/1355734.1355746.

[16] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model", *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009. DOI: 10.1109/TNN.2008.2005605.

[17] M. Al-Fares, S. Radhakrishnan, B. Raghavan, N. Huang, and A. Vahdat, "Hedera: Dynamic flow scheduling for data center networks", in *Proceedings of USENIX NSDI*, 2010, pp. 281–296. [Online]. Available: https://www.usenix.org/legacy/event/nsdi10/tech/full_papers/al-fares.pdf.

[18] A. R. Curtis, J. C. Mogul, J. Tourrilhes, P. Yalagandula, P. Sharma, and S. Banerjee, "DevoFlow: Scaling flow management for high-performance networks", in *Proceedings of ACM SIGCOMM*, 2011, pp. 254–265. DOI: 10.1145/2018436.2018466.

[19] C.-Y. Hong et al., "SWAN: Achieving high utilization in networks", in *Proceedings of ACM SIGCOMM*, 2013, pp. 3–14. DOI: 10.1145/2486001.2486012.

[20] S. Jain et al., "B4: Experience with a globally-deployed software defined WAN", in *Proceedings of ACM SIGCOMM*, 2013, pp. 3–14. DOI: 10.1145/2486001.2486019.

[21] I. F. Akyildiz, A. Lee, P. Wang, M. Luo, and W. Chou, "A roadmap for traffic engineering in SDN-openflow networks", *Computer Networks*, vol. 71, pp. 1–30, 2014. DOI: 10.1016/j.comnet.2014.06.002.

[22] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey", *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015. DOI: 10.1109/JPROC.2014.2371999.

[23] V. Mnih et al., "Human-level control through deep reinforcement learning", *Nature*, vol. 518, no. 7540, pp. 529–533, 2015. DOI: 10.1038/nature14236.

[24] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning", in *Proceedings of AAAI*, 2016, pp. 2094–2100. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/10295.

[25] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning", in *Proceedings of ACM HotNets*, 2016, pp. 50–56. DOI: 10.1145/3005745.3005750.

[26] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay", in *International Conference on Learning Representations*, 2016. [Online]. Available: https://arxiv.org/abs/1511.05952.

[27] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning", in *Proceedings of the International Conference on Machine Learning*, 2016, pp. 1995–2003. [Online]. Available: http://proceedings.mlr.press/v48/wangf16.html.

[28] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry", in *Proceedings of the International Conference on Machine Learning*, 2017, pp. 1263–1272. [Online]. Available: http://proceedings.mlr.press/v70/gilmer17a.html.

[29] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs", in *Advances in Neural Information Processing Systems*, 2017. [Online]. Available: https://arxiv.org/abs/1706.02216.

[30] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks", in *International Conference on Learning Representations*, 2017. [Online]. Available: https://arxiv.org/abs/1609.02907.

[31] H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with Pensieve", in *Proceedings of ACM SIGCOMM*, 2017, pp. 197–210. DOI: 10.1145/3098822.3098843.

[32] A. Valadarsky, M. Schapira, D. Shahaf, and A. Tamar, "Learning to route", *arXiv preprint arXiv:1708.03074*, 2017. arXiv: 1708.03074. [Online]. Available: https://arxiv.org/abs/1708.03074.

[33] P. W. Battaglia et al., "Relational inductive biases, deep learning, and graph networks", *arXiv preprint arXiv:1806.01261*, 2018. [Online]. Available: https://arxiv.org/abs/1806.01261.

[34] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html.

[35] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks", in *International Conference on Learning Representations*, 2018. [Online]. Available: https://arxiv.org/abs/1710.10903.

[36] Z. Xu et al., "Experience-driven networking: A deep reinforcement learning based approach", in *Proceedings of IEEE INFOCOM*, 2018, pp. 1871–1879. DOI: 10.1109/INFOCOM.2018.8485853.

[37] P. Almasan, J. Suarez-Varela, A. Badia-Sampera, K. Rusek, P. Barlet-Ros, and A. Cabellos-Aparicio, "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case", *arXiv preprint arXiv:1910.07421*, 2019. arXiv: 1910.07421. [Online]. Available: https://arxiv.org/abs/1910.07421.

[38] N. Jay, N. Rotman, B. Godfrey, M. Schapira, and A. Tamar, "A deep reinforcement learning perspective on internet congestion control", in *Proceedings of the International Conference on Machine Learning*, 2019, pp. 3050–3059. [Online]. Available: http://proceedings.mlr.press/v97/jay19a.html.

[39] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?", in *International Conference on Learning Representations*, 2019. [Online]. Available: https://arxiv.org/abs/1810.00826.

[40] S. Abbasloo, C.-Y. Yen, and H. J. Chao, "Classic meets modern: A pragmatic learning-based congestion control for the internet", in *Proceedings of ACM SIGCOMM*, 2020, pp. 632–647. DOI: 10.1145/3387514.3405892.

[41] K. Rusek, J. Suarez-Varela, P. Almasan, P. Barlet-Ros, and A. Cabellos-Aparicio, "RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN", *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 10, pp. 2260–2270, 2020. DOI: 10.1109/JSAC.2020.3000405.

[42] D. Pujol-Perich et al., "IGNNITION: Bridging the gap between graph neural networks and networking systems", *IEEE Network*, vol. 35, no. 6, pp. 171–177, 2021. DOI: 10.1109/MNET.001.2100266.

[43] M. Ferriol-Galmes et al., "RouteNet-Fermi: Network modeling with graph neural networks", *IEEE/ACM Transactions on Networking*, vol. 31, no. 6, pp. 3080–3095, 2023. DOI: 10.1109/TNET.2023.3269983.

[44] J. Suarez-Varela et al., "Graph neural networks for communication networks: Context, use cases and opportunities", *IEEE Network*, vol. 37, no. 3, pp. 146–153, 2023. DOI: 10.1109/MNET.001.2200430.

[45] A. Abrol, R. Kumar, S. K. Biswash, and N. Kumar, "A deep reinforcement learning approach for adaptive traffic routing in next-gen networks", *arXiv preprint arXiv:2402.04515*, 2024. arXiv: 2402.04515. [Online]. Available: https://arxiv.org/abs/2402.04515.